

File Handling

What is File?

- A file is a container for storing data on disk.
- Java uses the File class to work with files.
- Files can be created, deleted, or checked using File methods.
- Files can store text, numbers, or binary data.

Why File Handling is Required?

- To store data permanently instead of keeping it only in memory.
- To read and write data from/to files for later use.
- To share data between different programs or systems.
- To organize and manage large data efficiently.

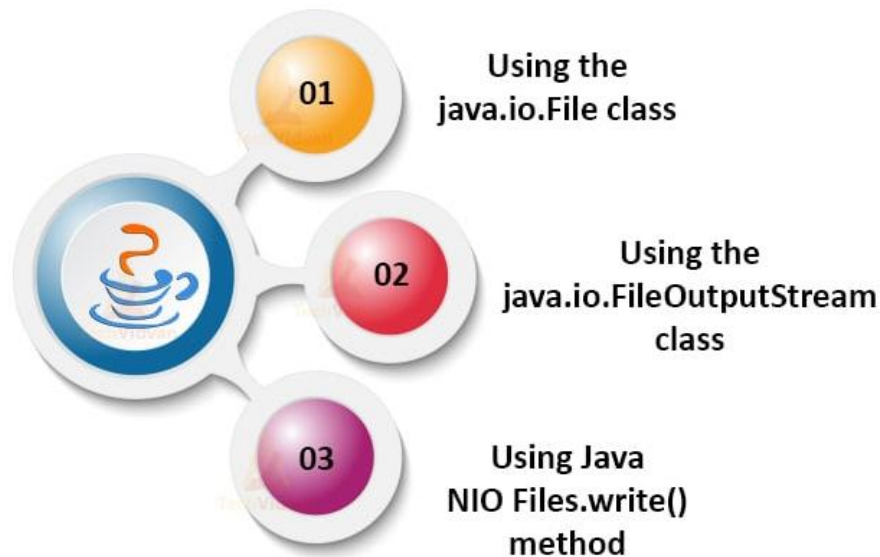
What is File Handling?

- File handling in Java refers to the process of performing operations on files such as:
 - Creating files
 - Existing Files
 - Deleting files
 - Reading files
 - Writing to files
- Files allow Java programs to store persistent data, which means data that remains even after the program terminates.
- It helps a program save and use information permanently on the computer.



1. Creating Files :-

- Use the File class to represent the file path.
- Call `createNewFile()` to create a new empty file.
- Returns true if created, false if it already exists.
- May throw `IOException` if the path is invalid or permission denied.



Example -

```
package file_handling;

import java.io.File;
import java.io.IOException;

public class file_created_or_not {

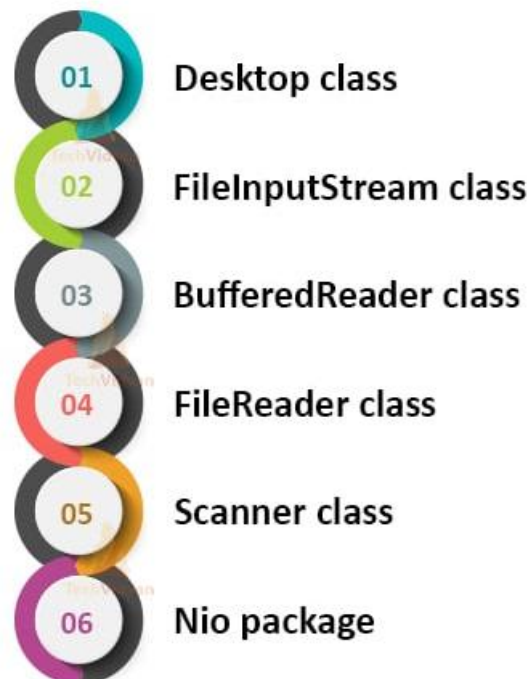
    public static void main(String[] args) throws IOException {
        File f=new File("C:\\Users\\sagar\\eclipse-workspace\\CoreJava\\first.txt");
        if(f.createNewFile())
        {
            System.out.println("File is Created");
        }
        else
        {
            System.out.println("File is Already Created!");
        }
    }
}
```

Output –

File is Created

2. Existing Files :-

- To open a file, first create a File object with the file path.
- Then use FileReader or Scanner to open and read the file.
- The file must exist and the program must have read permissions.



Example –

```
package file_handling;

import java.io.File;

public class file_exists_or_not {

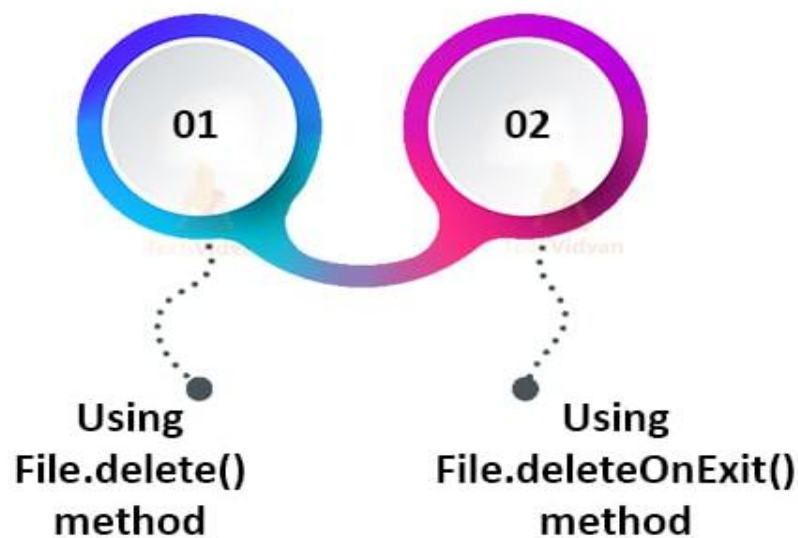
    public static void main(String[] args) {
        File f=new File("C:\\Users\\sagar\\eclipse-workspace\\CoreJava\\second.txt");
        if(f.exists())
        {
            System.out.println("File is Exists");
        }
        else
        {
            System.out.println("File is Not Exists!");
        }
    }
}
```

Output –

File is Exists

3. Deleting Files :-

- In Java, files can be deleted using the delete() method of the File class.
- The method removes the file permanently and returns true if successful, false if it fails.
- Deletion may fail if the file is in use, does not exist, or lacks permission.
- To delete a directory, it must be empty; otherwise, its contents must be deleted first.



Example –

```
package file_handling;

import java.io.File;

public class file_delete_or_not {

    public static void main(String[] args) {
        File f=new File("C:\\Users\\sagar\\eclipse-workspace\\CoreJava\\first.txt");
        if(f.delete())
        {
            System.out.println("File is Deleted");
        }
        else
        {
            System.out.println("File is Not Deleted!");
        }
    }
}
```

Output –

File is Deleted

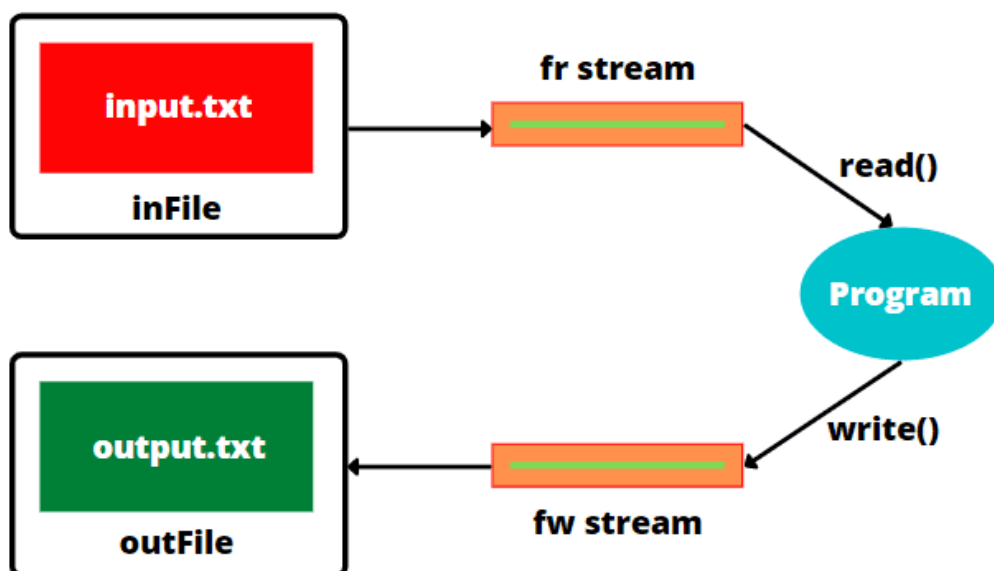
4. Reading & Writing Files :-

Reading Files :-

- Use FileReader or Scanner to open and read files.
- Reads data character by character (FileReader) or line by line (Scanner).
- Must handle exceptions like FileNotFoundException.
- Always close the file after reading using close().

Writing Files :-

- Use FileWriter or BufferedWriter to write data to files.
- Can create a new file or overwrite/append to an existing file.
- Must handle exceptions like IOException.
- Always close the writer to save data and free resources.



FileReader in Java :-

- The FileReader class is used to read data from files.
- It reads character data (not binary).
- It is part of the java.io package.
- It works only with text files.

Syntax:

```
FileReader fr = new FileReader("filename.txt");
```

Example –

```
package file_handling;

import java.io.File;
import java.io.FileReader;
import java.io.IOException;

public class file_reader {

    public static void main(String[] args) throws IOException {
        File f=new File("C:\\Users\\sagar\\eclipse-workspace\\CoreJava\\read.txt");
        f.createNewFile();

        FileReader fr = new FileReader(f);

        int i;

        while ((i = fr.read()) != -1) {
            System.out.print((char) i);
        }

        fr.close();
    }
}
```

Output –

Hello Sagar

FileWriter in Java :-

- The FileWriter class is used to write data into files.
- It writes character data (text) to the file.
- It is also part of the java.io package.
- It can create a new file or overwrite existing content.

Syntax:

```
FileWriter fw = new FileWriter("filename.txt");
```

Example –

```
package file_handling;

import java.io.File;
import java.io.FileWriter;
import java.io.IOException;

public class file_writer {

    public static void main(String[] args) throws IOException {
        File f = new File("C:\\Users\\sagar\\eclipse-workspace\\CoreJava\\write1.txt");
        f.createNewFile();

        FileWriter fw = new FileWriter(f);
        fw.write("Hello Pune");
        fw.close();

        System.out.println("File Created Successfully");
    }
}
```

Output –

File Created Successfully

File Class :-

- The File class is used to handle files and directories in Java.
- It ensures compatibility across different operating systems.
- It is part of the java.io package.
- It provides methods to create, delete, and check files or directories.

How to Creating File Object :-

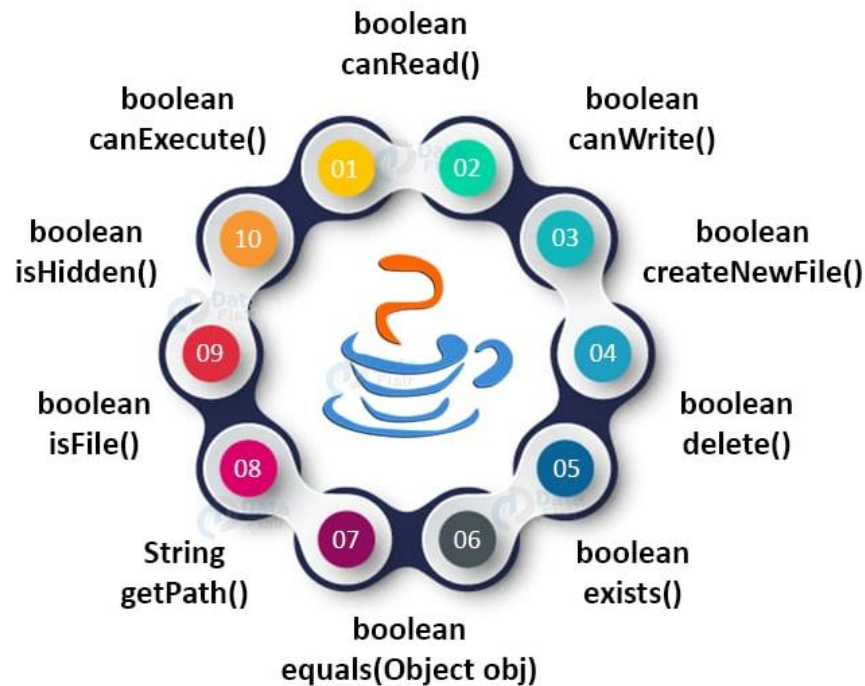
- The File object is created using the File class constructor.
- It represents the path of a file or directory.
- Creating a File object does not create the actual file, it only defines the path.

Syntax –

File objectName = new File("Directory or Path");

Methods of File Classes :-

1. **createNewFile()** – Creates a new file if it does not exist.
2. **exists()** – Checks if the file exists.
3. **delete()** – Deletes the file.
4. **canRead()** – Checks if the file is readable.
5. **canWrite()** – Checks if the file is writable.
6. **getName()** – Returns the name of the file.
7. **getPath()** – Returns the path of the file.
8. **length()** – Returns the size of the file in bytes.
9. **isFile()** – Checks if it is a file (not a directory).
10. **isDirectory()** – Checks if it is a directory.
11. **list()** – Returns an array of file names in a directory.
12. **mkdir()** – Creates a directory.
13. **mkdirs()** – Creates directories including any necessary parent directories.



Example –

```
package file_handling;

import java.io.File;
import java.io.IOException;

public class file_class {

    public static void main(String[] args) throws IOException {
        File f=new File("C:\\Users\\sagar\\eclipse-workspace\\CoreJava\\thursday.text");
        f.createNewFile();
        System.out.println("File is Created");
        System.out.println("canwrite: "+f.canWrite());
        System.out.println("canread: "+f.canRead());
        System.out.println("exits: "+f.exists());
        System.out.println("delete: "+f.delete());
        System.out.println("getabsoulatepath: "+f.getAbsolutePath());
        System.out.println("getname: "+f.getName());
        System.out.println("getpath: "+f.getPath());
        System.out.println("isfile: "+f.isFile());
        System.out.println("ishidden: "+f.isHidden());
        System.out.println("length: "+f.length());
        System.out.println("mkdir: "+f.mkdir());
        System.out.println("renameto: "+f.renameTo
            (new File("C:\\Users\\sagar\\eclipse-workspace\\CoreJava\\thursday2.text")));
        System.out.println("gettotalspace: "+f.getTotalSpace());

    }

}
```

Output –

```
File is Created
canwrite: true
canread: true
exists: true
delete: true
getabsoulatepath: C:\Users\sagar\eclipse-workspace\CoreJava\thursday.text
getname: thursday.text
getpath: C:\Users\sagar\eclipse-workspace\CoreJava\thursday.text
isfile: false
ishidden: false
length: 0
mkdir: true
renameto: true
gettotalspace: 0
```